perplexity

D-prompt.md

The first **priority** is `IsGradient`: it accepts an identically zero reported gradient for every function. That defect propagates into likelihoods, posteriors, objectives, and maximizer certification. Separately, the native LM models fail the unconditional quadratic laws, and `StepRule` does not determine the step it claims to characterize. The review below uses the frozen material only; it does not assume the absent host implementations behave as their comments promise.[1]

Numbers within each file refer to the prompt's nine review categories. Categories with no additional finding are grouped.

`probability.lth`

1. Laws that are wrong or have an invalid domain.

   ```
   ProfiledIsGaussianAtItsSigma ... (y : VecOf m.k)
   ```

   The domain includes zero residual sum of squares and `m.k == 0`. With one observation, `mean t = t`, and `y = t`, the construction evaluates `log 0`, divides by zero, and uses an infinite optimizing log-scale. There is no finite maximizing sigma: the full likelihood grows without bound as sigma tends to zero. State the profile construction and its comparison law on positive observation count and positive RSS, rather than making them total over the present domain.[1]

   ```
   UniformAtLevel ... := uniformPValue d vs >= l.alpha
   ```

   "it holds on a fraction 1 - alpha of calibrated samples"

   The predicate defines acceptance of this particular diagnostic, not exact calibration. Its tail uses a limiting distribution, a finite-sample correction, and a truncated numerical approximation. A counterexample to the exact fraction claim is the empty sample: `uniformPValue` is always `1`, so acceptance has probability one, not $1 - \alpha$. Moreover, `UnitList` supplies neither independence nor a sampling distribution. The certificate must name the actual sample generator and measured acceptance behavior.[1]

2. Laws that do not determine what they claim.

   ```
   else if nv == 0.0 then true
   ```

   Set `g t := zeros n`. `IsGradient` then passes for every `f`, every point, and every resolution, even when `f` is nonconstant.
   The nonzero branch also checks only the reported gradient's own direction:

   ```
   h := scale (r.step / nv) v
   ```

In exact arithmetic, take $f(t) = t_1 + t_2$ and report $g(t) = (1, 0)$. The measured slope along g is $1 = \|g\|$, although the true gradient is $(1, 1)$. Testing independently generated nonzero directions is necessary; removing the zero shortcut alone is insufficient. [1]

> `Likelihood ... := TangentShapes ... & ScoreIsGradient ... & InfoSymmetric ... & InfoNonNegative ...`

A record with arbitrary nonconstant `loglik`, identically zero `score`, and identically zero `info` passes all four laws, subject only to the required shapes. Even after repairing the score law, symmetry and nonnegativity do not identify Fisher information: replacing a genuine likelihood's information by zero still passes those two laws. The file explicitly postpones the sampling identities that would connect information to the distribution. [1]

> `KolmogorovOfMultiset ... d.distance vs == d.distance (reverseOnto vs nil)`

Reversing the outer list does not characterize a multiset: it does not test permutations within vectors, arbitrary permutations, or regrouping entries. `KolmogorovAgrees` currently closes this gap on `UnitList`, but then characterization depends on native agreement, rather than making agreement a corollary as the file claims. Outside `UnitList`, all laws permit arbitrary behavior despite the broader `distance` signature. [1]

3. Roles.
   No new `data` constructor appears here. Helpers including `normalCdf`, `normalTail`, `valueSums`, and `kolmogorovTail` have definitions but no attached characterization laws. Report these as helper-law omissions under D-2026-10-13-x, not as constructor-role violations. [1]

4. Models against their theories.
   The native `profiled` construction has the zero-RSS case in item 1. It therefore cannot supply the claimed total `Likelihood` on its current input domain. No host-model failure is established here. [1]

5. Against the cited decisions.

   > "characterized by its native definition exactly"

   `KolmogorovAgrees` is an essential conjunct, whereas the file's stated reading of 4.R10 makes native agreement a corollary of characterization. That is an internal contradiction with the cited design, not an objection to exact comparison itself. The series cutoffs below also contradict D-2026-10-14-c. [1]

6. Implication over a type.
   `IsGradient`'s zero branch makes the derivative assertion conditional on a property of the candidate answer. Do not merely restrict to nonzero reported gradients: that would leave true stationary points uncertifiable. Quantify the directional comparison over independently generated nonzero directions and retain all evaluation points.
   For profiling, use a subtype of model–data–parameter cases with positive count and positive RSS. [1]

7. Change first.
   Replace the self-direction gradient test with an independent-direction test. This repairs the largest shared certification hole before adding more laws downstream. [1]

8. Cutoffs.

```
tailSeriesTerms : Int := 100
tailSeriesFrom : Float64 := 0.2
```

These set numerical truncation and a branch resolution. Calling them “precision of the definition” does not exempt them from D-2026-10-14-c, which explicitly includes resolutions. Expose the requested accuracy/resolution and certify the approximation against it. `pi`, `sqrt2`, and the density’s numerical coefficients are mathematical constants, not acceptance cutoffs. [1]

9. Maximizer certification.
   The corrected independent-direction derivative law is a prerequisite for the nonquadratic certification described under `belief.lth`.

`probability_model.lut`

1–4. No verified host failure.

```
distance := extern "python:luthier.arrays.kolmogorov_distance"
```

The Python implementation is not supplied. In particular, the assertion that it performs the same floating-point operations in the same order cannot be checked from this route declaration. Important explicit cases are `nil`, a nonempty list of empty vectors, singleton endpoints, repeated values, and different regroupings of the same pooled entries. [1]

5–6. The model inherits `Kolmogorov`’s broad signature but only its `UnitList` certification domain. A certificate on unit-valued inputs must not silently certify every `List (Vec Float64)`. [1]

7. Change first: certify the host on the actual declared domain, including the nonempty-list/zero-pooled-count case.

8–9. No additional cutoff or maximizer finding.

`belief.lth`

1. Laws that are wrong or constructions whose result theory is not guaranteed.
   ```
   r.converged and 0.5 * c * dot d d <= s.tol
   ```

   `QuadraticConverges` quantifies over every `Settings`, including `maxIter = 1`. The comment’s “budget that is not tiny” is absent from the domain.
   The supplied native maximizer fails with $n = 1$, $b = 0$, $c = 1$, $x_0 = 1$, `tol = 0.01`, `maxIter = 1`, `floor = 0`, and damping `init = 1`, `factor = 2`, `least = 0.1`, `most = 10`. Its one step ends at $x = 0.5$, with decrement 0.125, and the run is exhausted. This is a valid budget-limited result but fails `QuadraticConverges`. [1]
   ```
   negate’s input theory is Posterior
   Posterior ... := PosteriorScoreIsGradient ... & PosteriorInfoSymmetric ...
   ```

   `Posterior` allows negative information. For example, `logp = 0`, `score = 0`, `info = -I` satisfies its stated laws but cannot become an `Objective`, which additionally requires a nonnegative step matrix. Thus the advertised conversion in `optimization.lth` is not guaranteed by this source theory. [1]

2. Laws that do not determine what they claim.

```
if r.converged then decrement ... <= s.tol else true
```

A truncated inverse can annihilate a nonzero score. Take

$$p(x_1, x_2) = x_2, qquadg = (0, 1), qquadM = operatornamediag(1, 0).$$

These satisfy the intended derivative and symmetry requirements. The decrement is zero everywhere, although the objective has no maximum and the score never vanishes. Returning the start with `converged = true` passes ascent and the purported stationarity check on this problem. Add a test for score components outside the retained subspace, or explicitly call this projected convergence rather than stationarity. [1]

> “the new mean is the maximum”

```
laplaceUpdate ... (laplaceMode ...).x
```

The update discards `converged`. A budget-exhausted result is consequently returned in the same form as an actual mode. Membership in `GaussianBelief` only checks the precision/density properties; it does not certify that the mean maximizes the posterior. Preserve or require the mode certificate before claiming a Laplace update at the maximum. [1]

```
precision := madd b.precision (l.info y m)
```

The file never ties this precision to the negative Hessian of `logp`. For example, a one-dimensional Gaussian likelihood with nonlinear mean $\mu(t) = t^2$, fixed unit sigma, and zero prior precision has Fisher information $4t^2$, but negative log-likelihood Hessian $6t^2 - 2y$. Thus this is an information-based Gaussian approximation in general, not a characterized Hessian-based Laplace approximation. [1]

3. Roles.
`MaxResult` is a record alias, not a new sum constructor. The meaning of `converged` is incompletely stated by item 2; that is a semantic-law gap, not a missing sum-role law. [1]

4. Models.
The explicit `lmMaximizer` counterexample belongs to `optimization_model.lut`, below. The host `lm` is absent and cannot be checked. [1]

5. Against the specification.

```
laplacePosterior ... (y : Vec Float64)
```

All four Laplace functions accept unrestricted data vectors, while the likelihood laws cover only `VecOf l.k`. Their claimed dependence on certified component laws does not justify use outside those laws’ domain. Restrict the data argument accordingly. [1]

6. Implication over a type.
`MaximizerStationary` can be vacuous when a model reports nonconvergence. For direct testing of that conditional property, generate cases whose returned result is converged. Separately test progress and success on an eligible problem family, so a model cannot avoid certification by always reporting failure.
`QuadraticConverges` needs an explicitly eligible settings/problem subtype, not an informal budget qualification. [1]

7. Change first.

Repair the convergence contract: distinguish projected decrement, actual stationarity, budget exhaustion, and a certified mode. Otherwise even a correctly implemented truncated solve can certify “convergence” where improvement remains possible. [1]

8. Cutoffs.
   `tol`, `maxIter`, and `floor` are already exposed. No additional literal numerical cutoff is established in this file.

9. Certifying the maximizer beyond quadratics.
   Add a nonvacuous success/optimality law over a generated nonquadratic family with a known maximum, not merely ascent and conditional decrement.
   One suitable family is
   $$p(x) = -sum_i left(fraca_i 2(x_i - b_i)^2 + fracd_i 4(x_i - b_i)^4 right), qquada_i > 0, d_i > 0,$$
   with analytic score and information chosen as the negative Hessian:
   $$g_i = -a_i(x_i - b_i) - d_i(x_i - b_i)^3, qquadM_ii = a_i + 3d_i(x_i - b_i)^2.$$
   Its unique maximum is b. Test that, for declared eligible settings, the maximizer reports convergence and meets the demand’s objective-gap and independent stationarity bounds at its returned point.
   Generate dimensions, centers, starts, and positive coefficient vectors from explicitly bounded, declared distributions. Include displaced starts, anisotropic coefficients, and cases where quartic terms matter. Supply budget, rank eligibility, and all accuracy bounds through the demand. This is a proposed additional law and generator; neither currently exists in the supplied files. Its certificate would cover that distribution, not arbitrary nonquadratic callbacks, consistent with L9.6 and L10. [1]

`belief_model.lut`

1–4. No verified implementation failure.

> “The host code is src/luthier/lm.py … it does not exist yet”

There is no specimen here to validate against the maximizer laws. The route declaration does not establish either quadratic or nonquadratic correctness. [1]

5, 8. Potential parameter-policy problem.

> “the damping schedule of the method is the host's own … not a setting of the maximizer”

If this means fixed host literals for the damping floor, ceiling, or schedule, it conflicts with P11 and the treatment of `Damping` in `optimization.lth`. The absent host code prevents establishing that implementation-level violation. [1]

6. No additional implication finding.

7. Change first: implement the specimen against the repaired maximizer contract, rather than treating the present declaration as evidence of certification.

8. Bind the additional generated nonquadratic tests to this actual host specimen and its dependency hash.

`optimization.lth`

1. Laws that are wrong.

   ```
   QuadraticStepAccepts ... norm (sub x0 b) == 0.0 or ... .accepted
   ```

   This is false for allowed floating-point cases. Set $n = 1$, $b = 0$, $c = 1$, $x_0 = 1$, `floor = 0`, and valid damping with `init = most = 1e20`, `least = 1`, `factor = 2`. The direction is approximately -10^{-20}. Adding it to `1.0` rounds back to `1.0`; the value does not strictly fall, and native `lmStep` rejects. The start is not the minimum. Eligibility must include representable decrease, or the law must allow this numerical outcome.[1]

   ```
   negate ... (p : Posterior n) : Objective n
   ```

   The negative-information posterior given above breaks the returned `Objective`'s nonnegativity law. Also, neither `Posterior` nor `Objective` explicitly requires an $n \times n$ matrix; symmetry alone accepts square matrices of the wrong dimension. Add shape laws and the appropriate positivity premise.[1]

2. Laws that do not determine what they claim.

   ```
   if s2.accepted then ... else ...
   ```

   `StepRule` never requires `s2.phi == o.value s2.x`. In the accepted branch a wrong model can report a fictitious decreased value.
   It also never requires accepting a trial that decreases the actual objective. A wrong step can use the prescribed direction but reject every nonquadratic trial, leaving point and value unchanged and updating damping/iteration exactly as the rejection branch requires. On the quadratic inputs it can behave correctly. Thus the laws neither determine the acceptance decision nor make step agreement a corollary.[1]

   ```
   (st : LMState n)
   ```

   Arbitrary states need not satisfy `st.phi == o.value st.x`, positive damping, or valid iteration count. Acceptance against a fabricated cached `phi` is not descent of the objective. State laws should operate on valid states, with preservation checked after stepping.[1]

3. Roles, L2.7.

   ```
   Reason = { running : Unit || converged : Unit || exhausted : Unit ||
   stalled : Unit }
   isConverged ... match ...
   ```

   `isConverged` is precisely the constructor-listing observer L2.7 excludes as a role law. `finished` gives a useful operational definition, but no Boolean laws state the roles of the four cases. Those laws should characterize its ordered decision: convergence first, then budget exhaustion, then excessive damping, otherwise running. In particular, `exhausted` must include failure of the earlier convergence condition.[1]

4. Models.
   `native : LMStep` fails `QuadraticStepAccepts` in item 1. No host failure is verified.

5. Against the cited design.

   > “the step's laws determine the step from the direction”

They do not, for the reasons in item 2. Consequently the claimed corollary of host/native step agreement does not follow. [1]

6. Implication over a type.
   Generate valid states, and separately generate decreasing and nondecreasing trial cases. Merely branching on the model's returned `accepted` flag lets the candidate control which obligation is tested. [1]

7. Change first.
   Complete `StepRule`: evaluate the candidate trial, determine acceptance from its actual value, and require the returned value and point to match that decision. This removes both fictitious descent and arbitrary rejection.

8. Cutoffs.
   `approxVec exact` expresses exact state copying or exact agreement with the same addition, rather than a hidden approximate-accuracy threshold. I would not parameterize genuine copying identities. Damping cutoffs are already fields of `Damping`. [1]

9. No separate maximizer family is required here; the additional nonquadratic law belongs to `Maximizer`.

`optimization_model.lut`

1, 4. Two explicit native-model failures.

```
    native : LMStep := { direction := dampedStep, step := lmStep }
```

Fails `QuadraticStepAccepts` at the large-damping, rounded-away-step case above.

```
    lmMaximizer (d : Damping) : Maximizer
```

Fails `QuadraticConverges` at the one-iteration case above: returned point `0.5`, decrement `0.125`, and `converged = false` for requested tolerance `0.01`. [1]

2, 5. The comment claiming step agreement follows from the laws is unsupported by the incomplete `StepRule`.

3, 6, 8–9. No additional finding.

7. Change first: correct the quadratic eligibility/success laws before presenting these native definitions as models. The first counterexample is numerical stagnation; the second is legitimate exhaustion, not necessarily an algorithm bug.

`arrays.lth`

1. Laws that are wrong for the intended rounded operations.
   ```
   RankUnderRotation ... (q : Orthogonal tol k) ... == ...
   ```
   Approximate orthogonality cannot guarantee exact retained-rank preservation. Take `tol = 1`,
   $$S = operatorname{diag}(1, 0.49), quad Q = operatorname{diag}(0.9, 1), quad fl = 0.5.$$
   `Q` meets the stated approximate orthogonality test, but

The retained rank changes from one to two because the threshold changes from `0.5` to `0.405`. Even nearly orthogonal matrices can cause this at a threshold boundary. Require a spectral margin sufficient for the perturbation, not exact integer invariance from approximate orthogonality alone.[1]

```
RankScales ... m.retainedRank fl (mscaleDef c s) == ...
```

Positive machine scaling may underflow eigenvalues to zero. For a one-dimensional matrix containing the smallest positive subnormal and scale `0.5`, the original rank is one and the scaled matrix is zero. This law needs a non-underflowing/non-overflowing domain and threshold stability.[1]

```
TraceAdditive ... within tol (...) (...)
```

Its flat result-based comparison can fail dramatically under cancellation. Use diagonals $A = (10^{16}, 1)$, $B = (-10^{16}, 1)$. Entrywise addition gives trace `2`, while separately rounded traces sum to `0`. A small tolerance on the final results cannot cover this error, despite its small size relative to the terms summed. The file already has magnitude-scaled comparison machinery; use it here.[1]

2. Laws that do not determine what they claim.

```
RankOfDiagonal ... (fl : ClearFloor) (d : ClearDiagonal)
```

The only explicit diagonal threshold oracle covers floors below `0.001` and diagonals far from the boundary. The projector identities and trace/rank agreement then check mutually consistent outputs, not independently which eigenvalues the floor should retain. At a PSD diagonal such as `diag(1, 0.01)` with `fl = 0.1`, retaining both directions satisfies the local projector, restriction, and trace/rank identities, although the intended answer retains one. This is a demonstrated local characterization gap, not a claim that this single assignment extends to a model satisfying the already-invalid rotation law everywhere.[1]

```
Arrays fields have unrestricted vector/matrix argument types
```

Most laws cover positive, compatible dimensions; spectral laws cover symmetric matrices. A candidate can agree on every covered input and behave arbitrarily on mismatched shapes or nonsymmetric `pinvSym` inputs. Narrow the operation signatures or explicitly bound the certificate domains.[1]

3. Roles.
   Arrays and matrices are record/function aliases. Their projections are covered by the settled product decision; no additional constructor-role violation is established.

4. Models.
   `arrays_model.lut` is not supplied. The mathematical retained-rank behavior described in the comments fails the first two laws above; actual host behavior remains unverified.[1]

5. Against the specification.
   The unrestricted scaling laws disregard 11.S4's distinction between rounded machine operations and real operations. Do not reopen that distinction; restrict these laws to where it supports the asserted relationship.[1]

6. Implication over a type.
   The file appropriately uses `PsdAt` rather than a rare implication. However, membership is determined by the candidate's own `retainedRank`. Generate known PSD or controlled-

spectrum matrices independently when certifying `pinvSym`, so incorrect rank behavior cannot exclude difficult cases from the tested subtype.[1]

7. Change first.
   Characterize retained rank directly on generated diagonals at every requested floor, including values on both sides of the threshold; then make rotation/scaling tests conditional on a quantified stability margin.
8. Cutoffs.

   ```
   ClearFloor ... f < 0.001
   ClearDiagonal ... 1.0 <= abs ... <= 1000.0
   ```

   These are fixed numerical eligibility cutoffs. Parameterize the floor bound and spectral range/separation through the demand, preferably stating separation relative to the requested floor rather than these three literals. Mathematical `0` and `1` in the rank-floor definition are not arbitrary numerical accuracy criteria.[1]
9. The maximizer's nonquadratic generator should include controlled conditioning and rank eligibility, so truncated array leaves cannot make its convergence test vacuous.

`experiment_zero.lut` — use-site consequences only

The experiment supplies `lmFloor`, checks convergence separately, and uses the Gaussian approximation in residual diagnostics. It also fixes

```
covFloor (n : Int) : RankFloor := toFloat64 n * doubles.epsilon
```

This is an internally chosen rank cutoff, unlike the demand-supplied LM floor. Under D-2026-10-14-c it should also come from the demand. I flag this as a use-site policy consequence, not a style review of the experiment. Its fitted, pooled residual transforms are not by themselves the independent uniform sample needed for the exact acceptance-fraction claim in `probability.lth`.[1]

Not sure

- The behavior of `luthier.arrays.kolmogorov_distance`, `arrays_model.lut`, and the missing `luthier.lm` specimen; their implementations are not supplied.
- How the interpreter handles overflow, division by zero, and nonfinite intermediate results when `Float64` is specified as finite. The profile constructions require such cases to be excluded or represented explicitly, regardless of that choice.
- Full type-checking consequences of dependent indices, refinements, and imported-name visibility; the prompt states that these files only parse.
- Exact obligations from chapters 4, 7, 10, and 12 beyond the excerpts and the files' own quotations. Findings about 4.R10 above identify contradictions with the interpretation explicitly claimed in these files.
- A certified distribution for arbitrary posterior callbacks. The proposed quartic generator establishes evidence only over its declared family and distribution, not over all objectives.

- Any certificate already issued, its grade, or its sampled domain. The supplied declarations and comments contain no such evidence. [1]

⁂

1. D-prompt.md